

Rapport – Ketchapp Circle

1. Introduction – Cahier des charges :

- Nous voulons réaliser un mini-jeu inspiré de Ketchapp Circle
- Un ovale se déplace le long d'une ligne brisée générée aléatoirement
- Le but du jeu est d'éviter de sortir de la ligne / de capturer des objets
- Le joueur peut cliquer sur l'écran pour faire monter l'ovale, qui redescend ensuite tout seul
- Un dessin de l'interface graphique du jeu (ou une capture d'écran de KC)

2a. Analyse globale :

Fonctionnalités principales :

1. Interface graphique avec l'ovale et la ligne brisée
2. Contrôle du mouvement de l'ovale
3. Génération et défilement automatique de la ligne brisée
4. Gameplay
5. Éléments graphiques
6. Menu du jeu (optionnel)

2b. Analyse détaillée :

- Fonctionnalité 1 :
 1. dessiner un ovale dans une fenêtre graphique
→ difficulté basse, priorité 1
 2. dessiner une ligne brisée dans une fenêtre graphique
→ difficulté moyenne, priorité 1
- Fonctionnalité 2 :
 1. faire monter l'ovale lorsqu'on clique
→ difficulté moyenne, priorité 1
 2. faire descendre l'ovale lorsqu'on ne clique pas
→ difficulté basse, priorité 1
 3. utiliser une vitesse pour créer un mouvement de saut
→ difficulté moyenne, priorité 2
- Fonctionnalité 3 :
 1. faire défiler la fenêtre pour faire avancer l'ovale
→ difficulté basse, priorité 1
 2. créer une ligne brisée infinie
→ difficulté moyenne, priorité 1

- Fonctionnalité 4 :
 1. mettre un score qui augmente à chaque seconde
→ difficulté basse, priorité 1
 2. système de perte de vie si le cercle touche la ligne
→ difficulté moyenne, priorité 1
 3. meilleur score
→ difficulté haute, priorité 3

- Fonctionnalité 5 :
 1. faire des oiseaux qui avancent à une vitesse différente
→ difficulté moyenne, priorité 2
 2. faire un arrière plan joli
→ difficulté basse, priorité 2
 3. faire une lune qui tourne sur elle-même (par réaliste mais fun)
→ difficulté moyenne, priorité 3
 4. ovale qui clignote quand il perd une vie
→ difficulté moyenne, priorité 2

- Fonctionnalité 6 :
 1. faire un menu avec un bouton « Play »
→ difficulté moyenne, priorité 3
 2. faire un menu de défaite qui affiche le score
→ difficulté moyenne, priorité 2
 3. possibilité de recommencer directement après la défaite
→ difficulté haute, priorité 3

3. Plan de développement :

Tâche :

1. apprentissage de Swing : 1h
2. apprentissage des Threads : 30mn
3. dessin de l'ovale : 15mn
4. événements souris : 15mn
5. thread de mouvement : 1h
6. vitesse pour effet rebond : 1h
7. apprentissage du modèle MVC et de la méthode de rédaction du rapport : 2h30
8. rédaction du document d'analyse et de conception: 1h
9. rédaction du reste du document : 5h
10. installation et mise en place de copilot : 30mn

11. dessin de la ligne brisée : 30mn
12. défilement de l'écran : 30mn
13. ligne brisée infinie : 1h
14. collision: 1h 30
15. score : 30 mn
16. apprendre à mettre des images : 15mn
17. vie (modélisation et affichage) : 1h
18. clignotement du cercle : 1h
19. oiseaux qui avancent (modélisation et affichage) : 1h30
20. affichage d'un arrière plan joli : 30mn
21. lune qui tourne sur elle même : 1h
22. « menu » de fin : 30mn

Diagramme de Gantt :

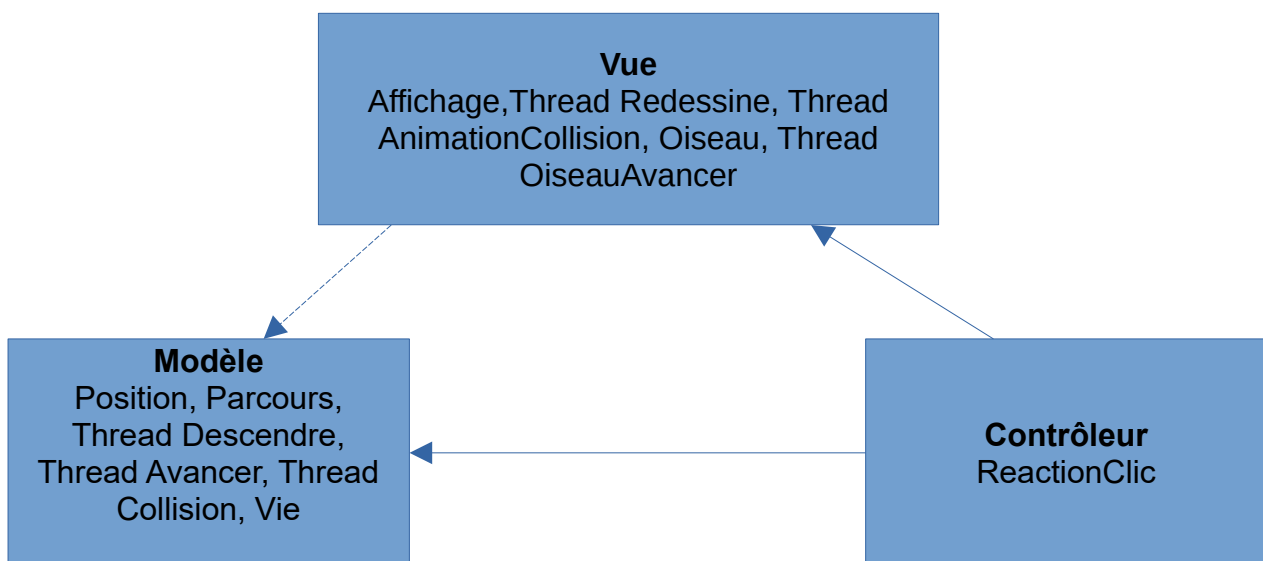
	Semaine 1				Semaine 2				Semaine 3				Semaine 4				Heures sup			
1)	x																			
2)		x																		
3)		x																		
4)		x																		
5)			x																	
6)				x																
7)					x	x	x													
8)							x													
9)									x			x		x	x					x
10)								x												
11)								x												
12)									x											
13)									x	x										
14)												x	x							
15)																				
16)													x							
17)														x		x				
18)																x				
19)																	x	x		
20)																		x		
21)																			x	
22)																			x	

4. Conception générale :

On va utiliser le patron MVC avec comme :

- Modèle :
 - création et position de l'ovale (Classe Position)

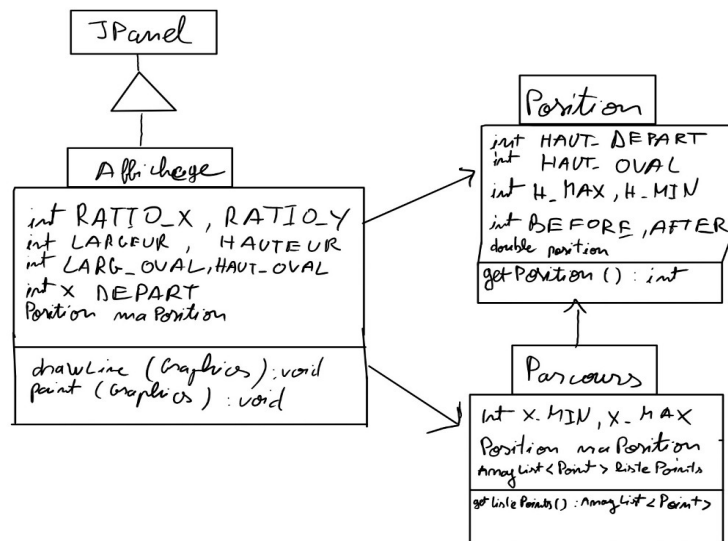
- création de la ligne brisée (Classe Parcours)
 - thread pour chute (Classe Descendre)
 - thread pour défilement et ligne infinie (Classe Avancer)
 - thread pour gérer la collision entre l'ovale et la ligne (Classe Collision)
 - création du système de vie (Classe Vie)
- **Vue :**
 - affichage de tous les éléments graphiques (Classe Affichage)
 - thread pour rafraîchir l'interface (Classe Redessine)
 - thread pour faire clignoter quand on perd une vie (Classe AnimationCollision)
 - créer des oiseaux et la lune (Classe Oiseau)
 - thread pour faire avancer les oiseaux et tourner la lune (Classe OiseauAvancer)
- **Contrôleur :**
 - Monte l'ovale sur clic de l'utilisateur (Classe ReactionClic)



4.1. Conception détaillée : Dessin de l'ovale et de la ligne brisée :

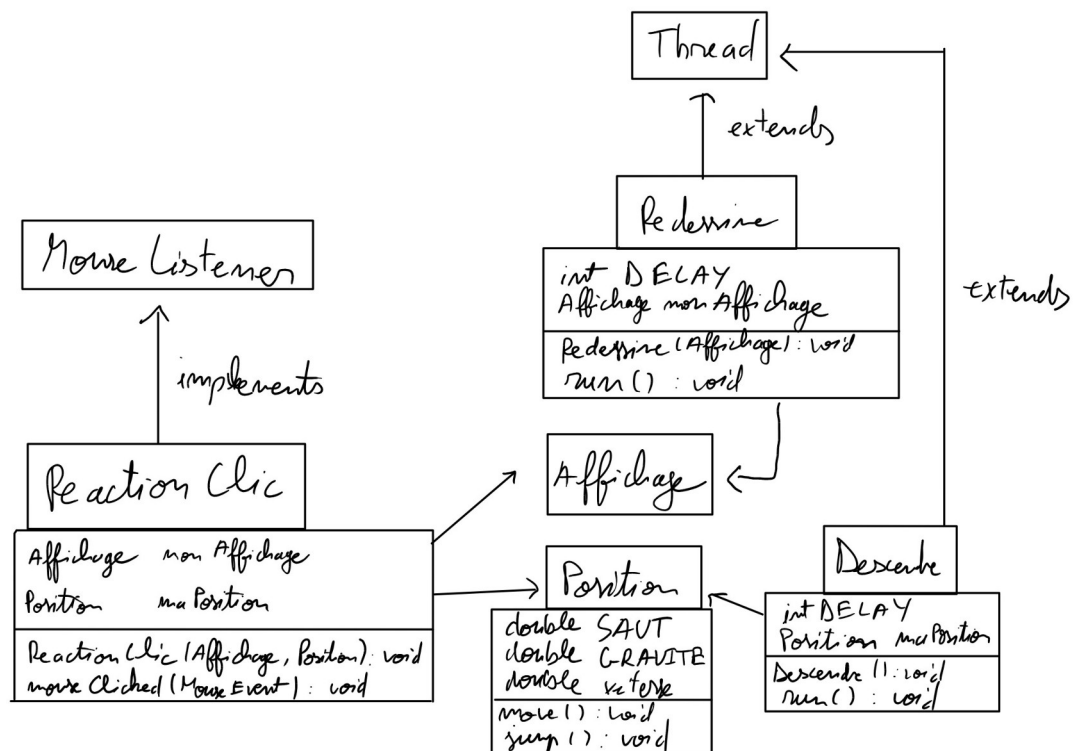
- Création de l'ovale dans la classe Position centrée sur sa position à elle :
 - HAUT_DEPART, HAUT_OVAL pour la position de l'ovale
 - H_MAX, H_MIN pour la hauteur maximale possible de l'ovale
 - BEFORE, AFTER pour l'horizon de l'ovale
- Création de la ligne brisée dans la classe Parcours qui contient la liste des points
 - X_MIN, X_MAX pour la hauteur maximale d'un point
- Dessin de l'ovale et de la ligne brisée dans la classe affichage :
 - Utilisation de l'API Swing et de la classe JPanel
 - RATIO_X, RATIO_Y pour les dimensions par rapport aux constantes dans Position
 - LARGEUR, HAUTEUR pour la taille de la fenêtre
 - LARG_OVAL, HAUT_OVAL, X_DEPART pour l'ovale par rapport aux ratios

- Diagramme de classe :



4.2. Conception détaillée : Mouvement de l'ovale :

- Classe **ReactionClic** pour gérer le clic
 - Programmation événementielle avec la classe **MouseListener**
 - Lie le clic sur la fenêtre d'**Affichage**
 - Appelle une fonction pour modifier la hauteur de l'ovale dans **Position**
- Thread de mise à jour de l'affichage dans la classe **Redessine**
 - **DELAY** pour un délai de 50 ms entre chaque rafraîchissement
 -
- Thread de chute automatique dans la classe **Descendre**
 - **DELAY** pour un délai de 50 ms entre chaque chute
 - Appelle une fonction pour modifier la hauteur de l'ovale dans **Position**
- Classe **Position** pour modifier la position de l'ovale
 - **SAUT** qui correspond à la hauteur du saut sur un clique
 - **GRAVITE** et vitesse pour gérer la chute avec un effet de rebond
- Diagramme de classe :



4.3 Conception détaillée : Génération et défilement automatique de la ligne brisée :

- Création de la ligne brisée dans la classe Parcours qui contient la liste des points
 - X_MIN, X_MAX pour la hauteur maximale d'un point
- Thread Avancer qui fait défiler et créer une ligne brisée infinie
 - DELAY de 100 ms
 - Fait avancer la position
 - Supprime les points derrière et crée les points suivants

Algorithme de génération de points :

Entrées :

- X_BEFORE : abscisse de début du parcours
- X_AFTER : abscisse de fin du parcours
- Y_DEPART : hauteur de départ de l'ovale
- HAUT_OVAL : hauteur de l'ovale
- ECART_MIN : écart horizontal minimum entre deux points
- ECART_MAX : écart horizontal maximum entre deux points
- Y_MIN : hauteur minimale possible
- Y_MAX : hauteur maximale possible

avec : $X_BEFORE < X_AFTER$; $ECART_MIN < ECART_MAX$; $Y_MIN < Y_MAX$

Sortie :

- LISTE(POINT)

Initialiser une liste vide L de points

//Ajouter à L un premier point :

abscisse = 0

ordonnée = Y_DEPART - HAUT_OVAL / 2

//Ajouter à L un second point :

abscisse = X_BEFORE + X_BEFORE / 20

ordonnée = Y_DEPART - HAUT_OVAL / 2

$X \leftarrow X_BEFORE + X_BEFORE / 20$

TANT QUE $X < X_AFTER$ FAIRE

$X \leftarrow X + \text{RANDOM}(\text{ECART_MIN}, \text{ECART_MAX})$

$Y \leftarrow \text{RANDOM}(Y_MIN, Y_MAX)$

Ajouter le point (X, Y) à la liste L

FIN TANT QUE

RETOURNER L

Algorithme de défilement de la ligne brisée :**Entrées :**

- LISTE(POINT) : liste ordonnée des points du parcours
- AVANCEMENT : avancement actuel du parcours
- X_BEFORE : décalage avant l'écran
- X_AFTER : limite de fin de l'écran
- ECART_MIN : écart horizontal minimum entre deux points
- ECART_MAX : écart horizontal maximum entre deux points
- Y_MIN : hauteur minimale possible
- Y_MAX : hauteur maximale possible

avec : $\text{ECART_MIN} < \text{ECART_MAX}$; $Y_MIN < Y_MAX$

Sortie :

- LISTE(POINT) mise à jour

Soit L la liste des points

Soit P2 le deuxième point de la liste L

SI $P2.x - \text{AVANCEMENT} + X_BEFORE < 0$ ALORS

Supprimer le premier point de la liste L

FIN SI

Soit Pdernier le dernier point de la liste L

SI $Pdernier.x - \text{AVANCEMENT} + X_BEFORE < X_AFTER$ ALORS

$X \leftarrow Pdernier.x + \text{RANDOM}(\text{ECART_MIN}, \text{ECART_MAX})$

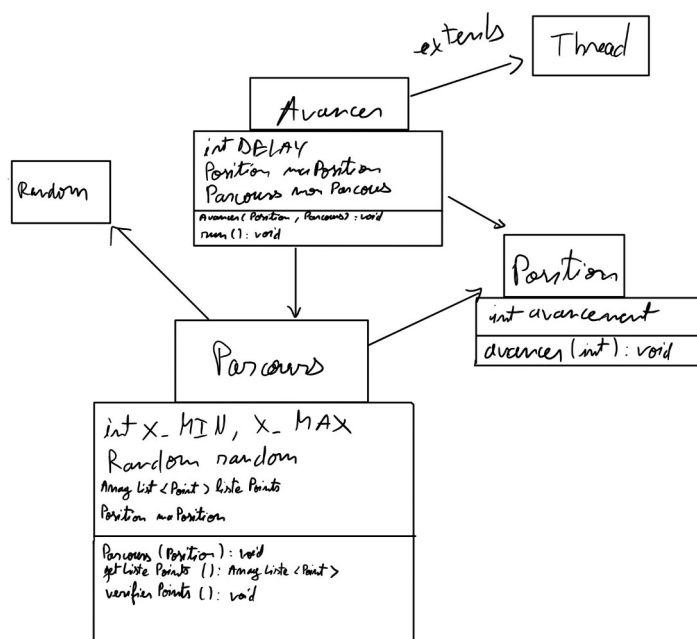
$Y \leftarrow \text{RANDOM}(Y_MIN, Y_MAX)$

Ajouter le point (X, Y) à la liste L

FIN SI

RETOURNER L

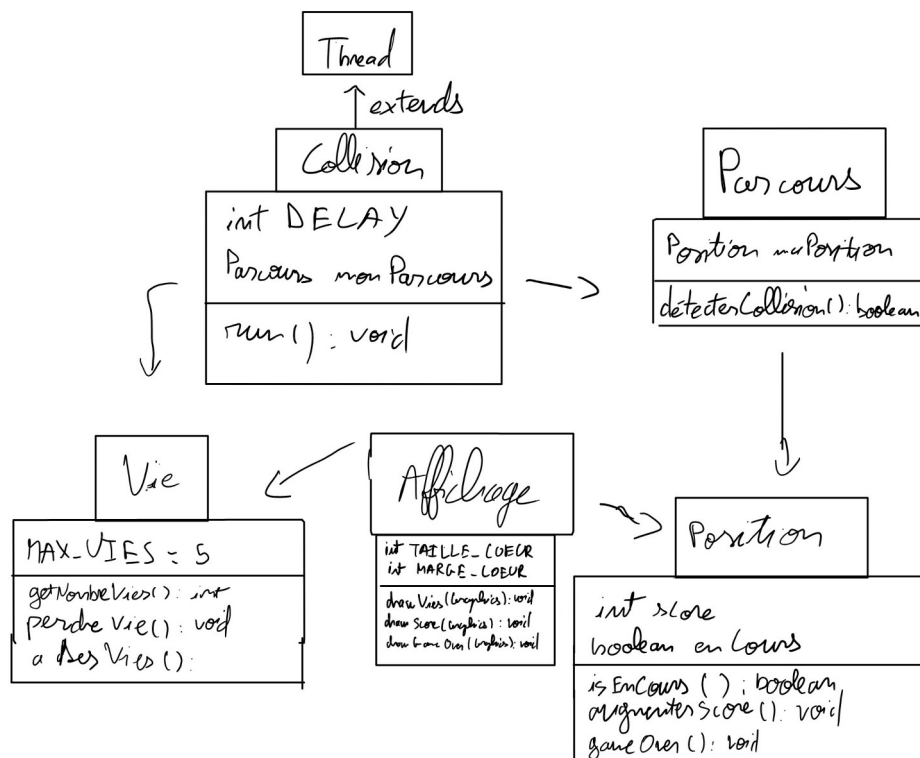
- Diagramme de classe :



4.4. Conception détaillée : Gameplay

- Gérer la collision dans la classe `Parcours`
 - Droite de la pente obtenue avec $PENTE = (Y2-Y1)/(X2-X1)$ donc hauteur de la droite obtenue avec $(X-X1)*PENTE$
- Score qui augmente toutes les secondes
 - Affichage pour l'afficher en bas à gauche
 - Stocké dans `Position`

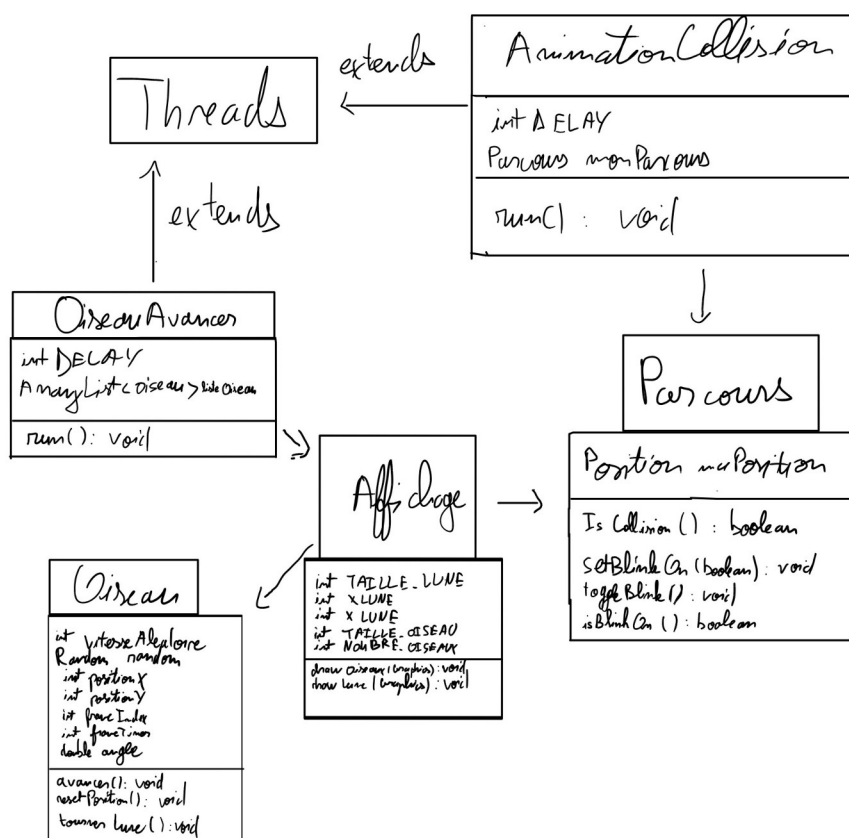
- Classe Vie pour gérer la vie de l'ovale
 - Image de coeur pour les afficher
- Thread Collision pour vérifier si l'ovale touche la ligne
 - Perdre une vie si touché
 - Si plus de vie, arrête le jeu
- Arrêt de tous les Thread si le jeu est fini
 - booléen enCours dans Position
 - image fixe finale
 - affiche un écran de défaite avec le jeu en fin et le score final
- Diagramme de classe :



4.5. Conception détaillée : Éléments graphiques :

- Thread AnimationCollision qui fait clignoter l'ovale quand il touche la ligne
 - variable blinkOn dans Parcours pour savoir si l'ovale clignote ou pas (pour alterner l'état)
- Arrière-plan « joli » créé dans la classe Affichage
 - Mode nuit : ciel bleu nuit et sol vert foncé

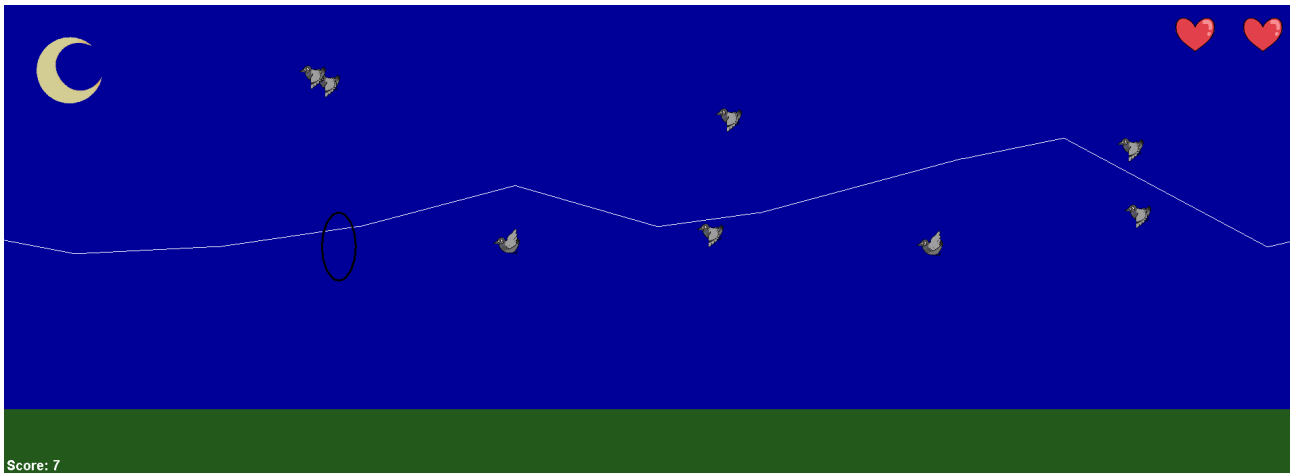
- Oiseaux qui avancent à une vitesse différente dans la classe Oiseau
 - Stocké dans le package Vue car aucun impact sur le gameplay
 - Avance à l'aide du Thread OiseauAvancer
 - 2 frames de l'oiseau qui alternent plus ou moins vite selon sa vitesse !
- Lune qui tourne sur elle-même (pour le fun) aussi stocké dans la classe Oiseau pour économiser une Classe et un Thread
 - Tourne à l'aide du Thread OiseauAvancer
 - Dessinée dans la classe Affichage et rotation grâce à Graphics2D.rotate
- Diagramme de Classe :



4.6. Conception détaillée : Menu du jeu :

- **Pas entièrement implémenté** : Pouvoir recommencer directement après avoir perdu est trop long et peu prioritaire.
- Texte de fin qui s'affiche par dessus le jeu et affiche le score final en grand
 - Si fin du jeu appelle la fonction `drawGameOver` de la classe `Affichage`

5. Résultats :



6. Documentation utilisateur :

- Prérequis : Java avec un IDE (ou Java tout seul si vous avez fait un export en .jar exécutable)
- Mode d'emploi (cas IDE) : Importez le projet dans votre IDE, sélectionnez la classe Main à la racine du projet puis cliquez sur l'icône « Run ».
- Mode d'emploi (cas .jar exécutable) : double-cliquez sur l'icône du fichier .jar
- Cliquez sur la fenêtre pour faire monter l'ovale.
- Cliquez sur la croix rouge pour fermer la fenêtre si vous avez perdu
- Réexécuter le programme pour recommencer à jouer

7. Documentation développeur :

- **Classe principale Main**
- **Packageux principaux suivant le patron MVC :**

- package model
 - classe principale Position
- package view
 - classe principale Affichage
 - dossier Images contenant toutes les images (entièrement dessinées par la meilleure artiste du monde : [ma copine](#))
- package controler
 - classe principale ReactionClic
- Constantes :
 - Constantes concernant l'ovale dans Position
 - Constantes concernant les points dessinant la ligne dans Parcours
 - Constante pour la vie dans Vie
 - Toutes les constantes pour l'affichage des éléments graphiques (Ovale, Vie, Lune, Oiseau) dans Affichage
- Prochaine étape : faire le menu de fin de jeu
 - ajouter une méthode reset() à toutes les classes nécessaires pour tout réinitialiser
 - modifier la classe ReactionClic pour appeler reset si on clique alors que le jeu est terminé

8. Conclusions et perspectives :

- J'ai vraiment fait du super boulot, je suis très fier notamment de la lune qui tourne sur elle-même et des battements d'ailes des oiseaux en fonction de leur vitesse
- J'ai eu du mal avec la collision entre la ligne et l'ovale, surtout sur la condition de collision, mais j'ai réussi en essayant des conditions au hasard
- Je suis très satisfait de mon projet, j'ai volontairement fait plus que nécessaire et dépassé sur le temps donné car j'adore coder en java et coder des jeux, je ne vois pas ça comme des devoirs
- J'ai hésité à faire le menu de fin pour relancer directement le jeu, mais je me suis dit que ça allait prendre trop de temps et j'étais déjà content de ce que j'ai fait

9. Retour d'expérience sur l'utilisation de Copilot :

Je ne connaissais pas avant, et Copilot est un très bon copilote. Il fait gagner un temps considérable sur des redondances : attribut, getter, setter.

En plus, comme je dois taper le prompt en commentaire, mon code en est rempli.

Grâce à lui, sur certaines méthodes, je n'ai même pas eu besoin de réfléchir à comment les implémenter, il suffisait de lui dire ce que je comptais faire et il donnait tout le code. J'avais juste à vérifier. Dans certains cas, je ne savais même pas comment les implémenter et grâce à lui j'y arrivais.

Cependant des fois il ne comprenait pas entièrement ou que la moitié du prompt. Comme il crache un gros bout de code, si je fais tab, ça écrit tout sauf que la moitié est fausse donc il faut effacer.

Il a aussi plus de mal à comprendre quand ça concerne les méthodes d'une autre classe, il sort des noms au hasard mais ça se corrige facilement.

Le défaut c'est qu'on ne peut pas « discuter » directement avec lui, si je veux comprendre quelque chose ou quoi. Mais je suppose que c'est normal car il est là pour nous aider à coder donc on est censé savoir ce que l'on fait.

J'ai essayé aussi Copilot Chat pour discuter, il reprend les modèles déjà existants (GPT, Gemini, etc) mais j'ai l'impression que c'est moins bien que les vrais. Mais un avantage est qu'on peut lui transmettre tous nos fichiers en 2-3 clics.

La méthode que j'utilise c'est que j'écris en commentaire ce que je veux faire et j'attends de voir ce qu'il me propose, si ce qu'il me propose a l'air bien je prends, sinon je réfléchis moi-même à l'implémentation.